

Optimizer Performance on Non-Convex Functions

Ankit Saini (2522MA01)
Tiyas Chakrabortty (2522MA02)
Tanu Goliyan (2621MA05)
Murari Ray (2621MA07)

Department of Mathematics
Indian Institute of Technology Patna

02-08-2026

1 Objective

The objective of this problem is to implement and compare five gradient-based optimization methods from scratch for two non-convex benchmark functions: **The two-dimensional Rosenbrock function** and a **One-dimensional simplified Ackley function**. The optimizers are tested at learning rates $\alpha = 0.01, 0.05$ and 0.10 . For every run, the final point which is obtained by the optimizer, the corresponding objective value, execution time and number of iterations are recorded and then the Convergence behaviour is visualized using interactive loss-versus-iteration plots.

The stopping rule of the convergence, used in the implementation is $\|\nabla f\| < 10^{-4}$ or a maximum of 1000 iterations, whichever occurs first. The purpose is not only to obtain a small objective value, but also to study how the choice of optimizer choice could affect stability, convergence speed.

2 Mathematical Formulation

2.1 Rosenbrock Function

For $a = 1$ and $b = 100$, the Rosenbrock function is defined by,

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

Its global minimum is at $(x, y) = (1, 1)$, where $f(1, 1) = 0$.

2.2 Simplified Ackley Function (1D)

The one-dimensional objective used in the experiment is

$$f(x) = -20 \exp(-0.2\sqrt{x^2}) - \exp(\cos(2\pi x)) + 20 + e.$$

The global minimum is at $x = 0$ with $f(0) = 0$.

3 Analytical Gradients

For Rosenbrock, the analytical gradient implemented in the code is

$$\frac{\partial f}{\partial x} = -2(1 - x) - 400x(y - x^2), \quad \frac{\partial f}{\partial y} = 200(y - x^2).$$

For the simplified Ackley function, the analytical derivative is implemented directly from the two x -dependent terms. The returned gradients are clipped to the interval $[-10^4, 10^4]$. This clipping is a numerical-stability measure intended to prevent extremely large gradients from immediately producing floating-point overflow or catastrophic steps.

4 Optimization Methods

We compared Five methods . Each method receives the analytical gradient, starting point, learning rate, maximum iteration count and tolerance.

4.1 Gradient Descent

Gradient Descent iteratively updates the current solution by moving in the negative gradient direction:

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$$

where θ_t is the current position at iteration t . For a curved non-convex valley, a large α can make the algorithm overshoot or diverge.

4.2 Gradient Descent with Momentum

The Momentum maintains a velocity term that accumulates information from previous gradients. This can accelerate movement in consistent directions, but excessive momentum, combined with a large learning rate can also produce oscillation or divergence.

4.3 Adam

This method maintains an exponentially weighted estimates of the first and second moments of the gradient. Therefore it adapts the effective step size coordinate-wise and is often more stable than plain gradient descent on difficult objectives.

4.4 RMSprop

RMSprop scales the gradient using a moving estimate of its squared magnitude. This reduces the effect of very large gradients and can be useful on objectives with different curvature scales.

4.5 AdaGrad

AdaGrad accumulates squared gradients and decreases the effective learning rate over time. It can converge rapidly on some problems, although its accumulated scaling can also make later updates very small.

5 Experimental Setup

The benchmark uses the same starting point for every optimizer at each function and learning rate. This makes the comparison fair, as any differences in performance come from the optimization method and learning rate, rather than from different starting points or random sampling.

Parameter	Setting
Objective functions	Rosenbrock (2D), simplified Ackley (1D)
Optimizers	Gradient Descent, Momentum, Adam, RM-Sprop, AdaGrad
Learning rates	0.01, 0.05, 0.10
Starting point: Rosenbrock	$[-1.5, 1.5]$
Starting point: Ackley	$[2.5]$
Tolerance	$\ \nabla f\ < 1 \times 10^{-4}$
Maximum iterations	1000
Timing	<code>time.perf_counter()</code>
Plot scale	Logarithmic y-axis

6 Results: Rosenbrock Function

All Rosenbrock function reached the maximum of 1000 iterations. Thus, none of the five methods satisfied the gradient-norm stopping threshold within the allowed limit for this starting point and the tested learning rates. The final values therefore represent the points reached after 1000 updates rather than verified global minima.

6.1 Results for $\alpha = 0.01$

Optimizer	Final x	Final y	$f(x, y)$	Time (s)	Iters
Gradient Descent	126.1185	648.7471	2.327801e+10	0.049054	1000
SGD with Momentum	235.4990	1000.0000	2.965870e+11	0.063694	1000
Adam	-0.9859	0.9778	3.947162e+00	0.073236	1000
RMSprop	0.8411	0.6935	4.474222e-02	0.043642	1000
AdaGrad	-1.2981	1.6910	5.284803e+00	0.051513	1000

Table 1: Rosenbrock results at $\alpha = 0.01$.

6.2 Results for $\alpha = 0.05$

Optimizer	Final x	Final y	$f(x, y)$	Time (s)	Iters
Gradient Descent	521.2500	500.0000	7.355029e+12	0.106472	1000
SGD with Momentum	-60.4470	1000.0000	7.042925e+08	0.039847	1000
Adam	0.7514	0.5638	6.187155e-02	0.067167	1000
RMSprop	0.7023	0.4342	4.372389e-01	0.064017	1000
AdaGrad	-1.2050	1.4582	4.865914e+00	0.045263	1000

Table 2: Rosenbrock results at $\alpha = 0.05$.

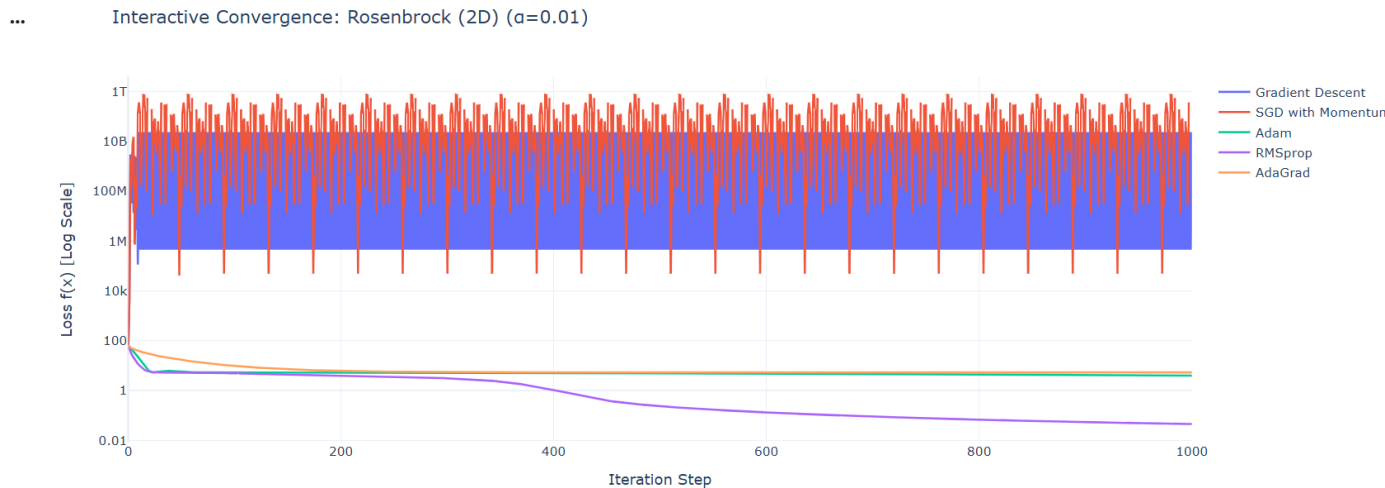
6.3 Results for $\alpha = 0.1$

Optimizer	Final x	Final y	$f(x, y)$	Time (s)	Iters
Gradient Descent	-956.0000	1000.0000	8.334521e+13	0.045310	1000
SGD with Momentum	1000.0000	1000.0000	9.980010e+13	0.057884	1000
Adam	0.9539	0.9098	2.124739e-03	0.067185	1000
RMSprop	0.3000	0.0150	1.052500e+00	0.133999	1000
AdaGrad	-1.0093	1.0245	4.040617e+00	0.213357	1000

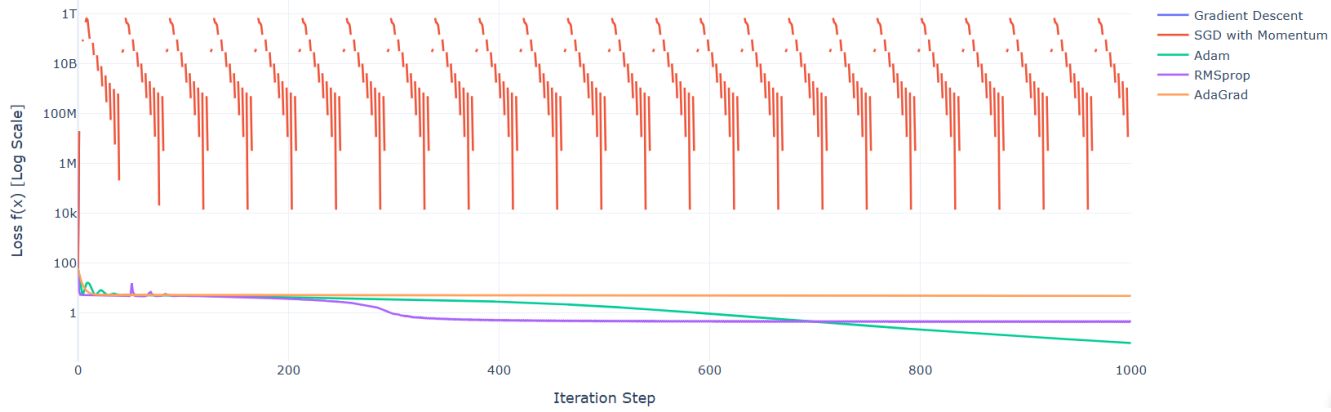
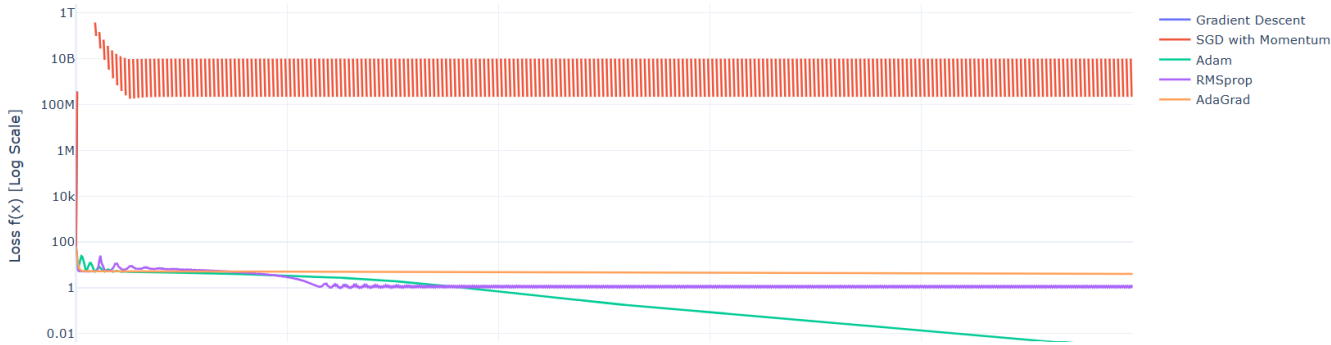
Table 3: Rosenbrock results at $\alpha = 0.1$.

7 Rosenbrock Convergence Plots

The following plots are the convergence outputs produced by the notebook. The vertical axis uses a logarithmic scale because the objective values span many orders of magnitude. Very large values caused by unstable runs can therefore dominate the visual scale.



...

Interactive Convergence: Rosenbrock (2D) ($\alpha=0.05$)Interactive Convergence: Rosenbrock (2D) ($\alpha=0.1$)

8 Results: Simplified Ackley Function

The Ackley results shows a different behaviour from Rosenbrock. Several optimizers terminate before 1000 iterations because the gradient-norm threshold is reached, but they often stop at local stationary points rather than the global minimum $x = 0$. although this is expected for a non-convex function and shows why the final point returned by an optimizer should not automatically be interpreted as the global optimum.

8.1 Results for $\alpha = 0.01$

Optimizer	Final x	$f(x)$	Time (s)	Iters
Gradient Descent	1.9745	6.559645e+00	0.003995	12
SGD with Momentum	-0.9685	3.574452e+00	0.122490	249
Adam	1.9745	6.559645e+00	0.126140	275
RMSprop	1.9745	6.559645e+00	0.012260	67
AdaGrad	1.9763	6.559819e+00	0.327585	1000

Table 4: Ackley results at $\alpha = 0.01$.

8.2 Results for $\alpha = 0.05$

Optimizer	Final x	$f(x)$	Time (s)	Iters
Gradient Descent	0.4236	3.931051e+00	0.386220	1000
SGD with Momentum	109.8839	2.601198e+01	0.335391	1000
Adam	1.9745	6.559645e+00	0.056508	248
RMSprop	1.9481	6.593224e+00	0.105489	1000
AdaGrad	1.9745	6.559645e+00	0.019284	115

Table 5: Ackley results at $\alpha = 0.05$.

8.3 Results for $\alpha = 0.1$

Optimizer	Final x	$f(x)$	Time (s)	Iters
Gradient Descent	0.0473	3.050312e-01	0.181852	1000
SGD with Momentum	-237.8358	2.104775e+01	0.162366	1000
Adam	1.9745	6.559645e+00	0.046462	224
RMSprop	1.9185	6.706120e+00	0.199885	1000
AdaGrad	1.9745	6.559645e+00	0.002992	27

Table 6: Ackley results at $\alpha = 0.10$.

9 Ackley Convergence Plots

The Ackley plots further show the sensitivity of the methods to learning rate. At $\alpha = 0.01$, Gradient Descent reaches a stationary point quickly, while Momentum exhibits damped oscillations. At $\alpha = 0.05$ and 0.10 , some methods become unstable or settle at higher objective values.

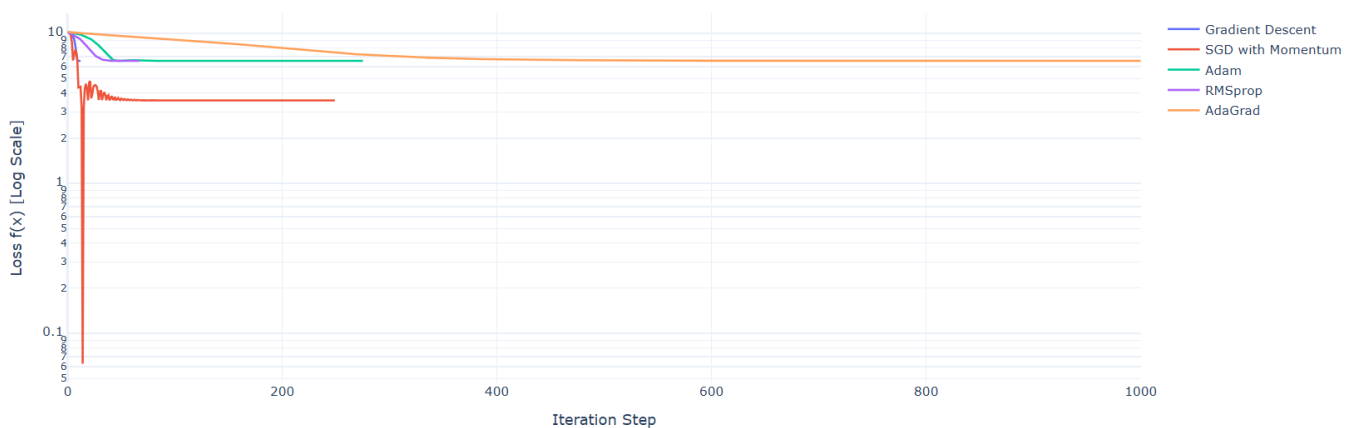


Figure 1: Ackley: 1D(0.01)

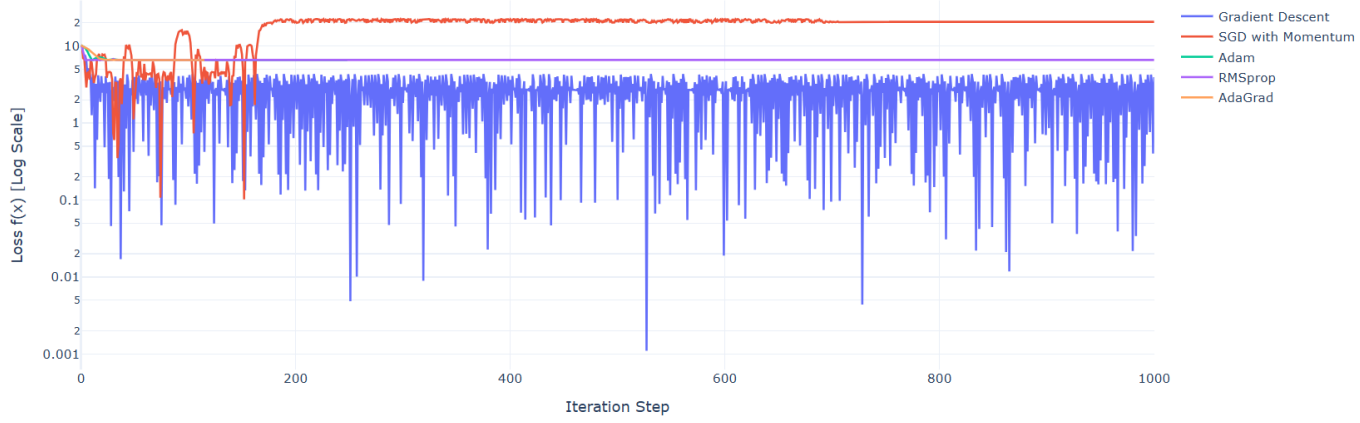


Figure 2: Ackley: 1D(0.05)

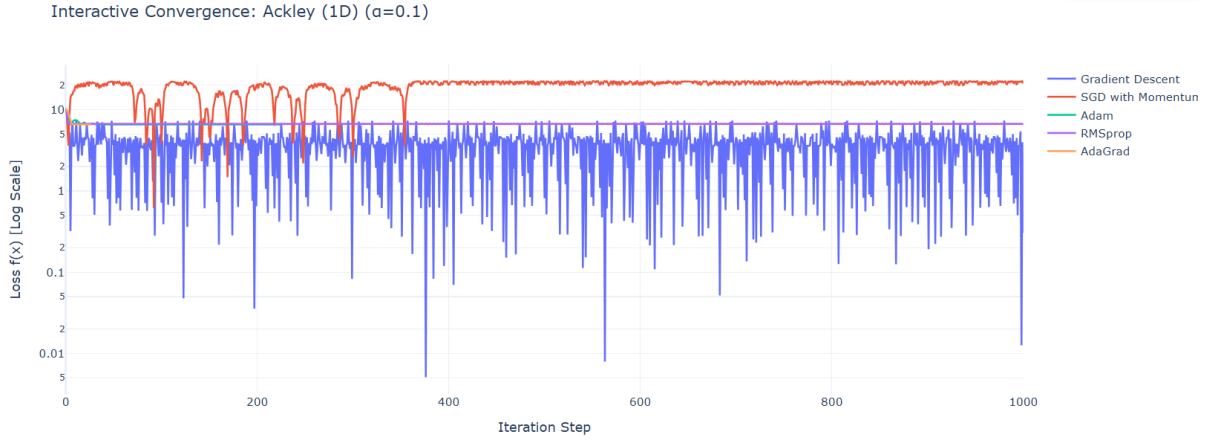


Figure 3: Ackley: 1D(0.1)

10 Comparative Analysis

10.1 Best Final Objective Value for Each Learning Rate

Function	α	Best final f	Optimizer	Final point
Rosenbrock	0.01	4.474222e-02	RMSprop	(0.8411, 0.6935)
Rosenbrock	0.05	6.187155e-02	Adam	(0.7514, 0.5638)
Rosenbrock	0.10	2.124739e-03	Adam	(0.9539, 0.9098)
Ackley	0.01	3.574452e+00	SGD with Momentum	$x = -0.9685$
Ackley	0.05	3.931051e+00	Gradient Descent	$x = 0.4236$
Ackley	0.10	3.050312e-01	Gradient Descent	$x = 0.0473$

Table 7: Best final objective among the five optimizers for each learning rate.

10.2 Rosenbrock Comparison

At $\alpha = 0.01$, RMSprop gives the smallest final objective value among the tested methods, $f \approx 4.47 \times 10^{-2}$. At $\alpha = 0.05$ and $\alpha = 0.10$, Adam gives the smallest final objective

values, approximately 6.19×10^{-2} and 2.12×10^{-3} , respectively. In contrast, plain Gradient Descent and Momentum become extremely unstable as the learning rate increases. Their final coordinates move far outside the Rosenbrock valley and the objective values become extremely large.

A notable point is that RMSprop at $\alpha = 0.10$ reaches (0.3000, 0.0150), but its objective value is about 1.05, which is worse than Adam at the same learning rate. Thus, a visually smooth curve does not necessarily imply the best final objective.

10.3 Ackley Comparison

For $\alpha = 0.01$, Momentum produces the lowest final objective among the five methods, $f \approx 3.57$ at $x \approx -0.9685$, but this is still far above the global minimum $f(0) = 0$. At $\alpha = 0.05$, Gradient Descent gives the lowest reported value, $f \approx 3.93$ at $x \approx 0.4236$. At $\alpha = 0.10$, Gradient Descent improves substantially to $f \approx 0.305$ at $x \approx 0.0473$, which is the closest result to the global minimum among the displayed runs.

Adam, RMSprop and AdaGrad frequently converge to points around $x \approx 1.9$ – 2.0 , where the Ackley objective is about 6.56–6.71. These are stationary/local-minimum regions of the non-convex landscape, illustrating that satisfying a gradient-norm stopping criterion does not guarantee global optimality.

11 Convergence Speed and Time

The measured execution time depends on both the number of iterations and the amount of numerical work performed by each optimizer. For Ackley at $\alpha = 0.01$, plain Gradient Descent is particularly fast in the reported run, terminating after only 12 iterations and taking about 0.004 s. RMSprop takes about 0.012 s for 67 iterations, while Momentum and Adam require 249 and 275 iterations, respectively.

At $\alpha = 0.10$ on Ackley, AdaGrad terminates after only 27 iterations and records the smallest runtime, about 0.003 s, although its final objective is not close to the global minimum. This shows an important distinction: fast termination is not the same as good optimization.

For Rosenbrock, every optimizer reaches the 1000-iteration limit for all three learning rates. Consequently, the timing differences there mainly reflect the computational cost of the optimizer updates rather than successful early convergence.

12 Impact of the Learning Rate

The learning rate α is the main hyperparameter varied in this experiment. The results show a strong sensitivity to α , especially for the non-convex Rosenbrock function.

$\alpha = 0.01$: The smaller step size is comparatively conservative. Rosenbrock remains difficult and all methods require 1000 iterations, but RMSprop obtains a small objective value. On Ackley, several methods reach stationary points within 12–275 iterations.

$\alpha = 0.05$: The larger step size improves Adam’s Rosenbrock result relative to $\alpha = 0.01$, but plain Gradient Descent and Momentum become highly unstable. For Ackley, Momentum reaches $x \approx 109.88$ with $f \approx 26.01$, showing severe overshooting.

$\alpha = 0.10$: This rate is too large for plain Gradient Descent and Momentum on Rosenbrock, where objective values become enormous. Adam remains comparatively

stable and obtains the best Rosenbrock result in the displayed $\alpha = 0.10$ experiment. For Ackley, however, plain Gradient Descent moves very close to the global minimum, giving $x \approx 0.0473$ and $f \approx 0.305$.

Therefore, there is no single learning rate that is best for every optimizer and every objective. The suitable learning rate depends on the geometry of the function and on the update rule.

13 Observations and Inferences

1. Rosenbrock is harder for the tested methods from the chosen starting point. Every Rosenbrock run uses the full 1000-iteration limit, and several fixed-step methods diverge.
2. Adam is the strongest overall Rosenbrock performer in the reported experiments at $\alpha = 0.05$ and $\alpha = 0.10$, while RMSprop is best at $\alpha = 0.01$ in terms of final objective value.
3. Momentum is not automatically more stable than ordinary gradient descent. Its accumulated velocity can amplify large steps when the learning rate is too high.
4. Ackley shows the difference between local and global optimization. Several methods terminate around $x \approx 1.97$ with $f \approx 6.56$, even though the known global minimum is $x = 0$, $f = 0$.
5. The smallest runtime or smallest iteration count should not be used alone to select an optimizer. For example, AdaGrad is very fast at $\alpha = 0.10$ on Ackley but stops at a value around 6.56 rather than the global minimum.
6. The logarithmic y-axis is useful because unstable runs produce objective values spanning many orders of magnitude. However, a log axis cannot represent exactly zero; consequently, the exact Ackley value $f(0) = 0$ would not be visible as a normal point on such a plot.
7. The convergence curves with very large spikes or repeated vertical-looking segments are evidence of unstable numerical updates rather than successful convergence.

14 Limitations

The experiment uses only one initial point for each objective. Different initial points can lead to different local minima on a non-convex function. Therefore, the reported final points should be interpreted as results for the specified initial conditions, not as proof that an optimizer always finds the same solution.

The benchmark also uses a fixed maximum of 1000 iterations and a single gradient-norm threshold. Some algorithms may require more iterations to reach a better region. Conversely, a gradient-norm criterion can stop at a local stationary point.

The supplied implementation clips analytical gradients to $\pm 10^4$ and filters non-finite or extremely large loss values when preparing the plots. These numerical safeguards improve robustness and plotting, but they also mean that the plotted curve is a numerically

protected representation of the raw optimization process rather than an unrestricted mathematical trajectory.

Finally, the timing values are machine- and implementation-dependent. They are most useful for relative comparison within the same execution environment.

15 Conclusion

The two non-convex functions were tested using five optimization methods: Gradient Descent, Gradient Descent with Momentum, Adam, RMSprop, and AdaGrad, with learning rates 0.01, 0.05, and 0.10. The final point, function value, execution time, iterations, and convergence plots were recorded for all 30 runs. The results show that optimizer performance depends strongly on the learning rate and the function. For Rosenbrock, larger learning rates caused Gradient Descent and Momentum to become unstable, while Adam showed better stability. For Ackley, several methods converged to local minima instead of the global minimum. The best reported Ackley result was obtained by Gradient Descent with $\alpha = 0.10$, giving $x \approx 0.0473$ and $f(x) \approx 0.305$.

Overall, the experiment shows that both final objective value and convergence behaviour are important when comparing optimization methods, rather than considering only runtime or iteration count.